

Chapter 7 Doubly Linked Lists

Doubly Linked Lists (DLLs)

◆ PART A: Conceptual Understanding

? Prompt

Define a doubly linked list and explain how it differs from a singly linked list.

✓ Answer

A **doubly linked list** is a linked list where **each node contains**:

1. **Element** → the data
2. **Next** → reference to the next node
3. **Prev** → reference to the previous node

Differences from singly linked list (SLL):

Feature	SLL	DLL
Pointers	Next only	Next & Prev
Remove Tail	$O(n)$	$O(1)$
Remove arbitrary node	$O(n)$	$O(1)$ (if node reference known)
Insert/delete anywhere	$O(n)$	$O(1)$

Advantage:

Allows insertion and deletion at **both ends and any arbitrary position** in $O(1)$ time.

? Prompt

Why is deleting the last node in a singly linked list inefficient?

✓ Answer

- SLL nodes only point forward
- To update the second-last node's `next` pointer to `None`, we must **traverse from the head**
- This takes **$O(n)$ time**

DLL solves this by storing a **prev pointer**, giving direct access to the previous node.

◆ PART B: Sentinel Nodes

? Prompt

What are header and trailer sentinels in a doubly linked list?

✓ Answer

- **Sentinels** are special “dummy” nodes at both ends of the list:
 - **Header** → beginning of list
 - **Trailer** → end of list
- They **do not store data**
- They simplify insertions and deletions because **every node has neighbors**
- Eliminates edge-case handling for empty or single-element lists

? Prompt

Explain the structure of an empty DLL with sentinels.

✓ Answer

- `header.next` → points to trailer
- `trailer.prev` → points to header
- All other fields irrelevant (`None`)
- `Size = 0`

`header ↔ trailer`

No real data nodes exist yet.

? Prompt

Describe the structure of a DLL with elements.

✓ Answer

- Nodes containing data are **inserted between header and trailer**
- `header.next` → first data node
- `trailer.prev` → last data node

Example sequence { JFK, PVD, SFO }:

`header ↔ JFK ↔ PVD ↔ SFO ↔ trailer`

◆ PART C: Advantages of Using Sentinels

? Prompt

What are the advantages of using header and trailer sentinel nodes?

✓ Answer

Simplifies insertion and deletion

- No special cases for empty list
- All insertions are between two nodes

Consistency

- Header and trailer never change
- Only internal nodes are updated

Simpler code

- No need to check for `None` when inserting at ends

◆ PART D: Node Structure in DLL

? Prompt

Describe the instance variables of a node in a doubly linked list.

✓ Answer

Field	Purpose
<code>_element</code>	Data stored
<code>_next</code>	Reference to next node
<code>_prev</code>	Reference to previous node

Diagram: Doubly Linked List with Sentinels

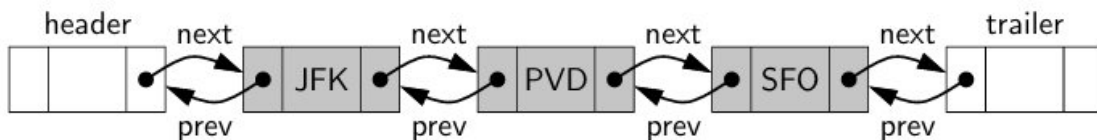


Figure 7.10: A doubly linked list representing the sequence { JFK, PVD, SFO }, using sentinels header and trailer to demarcate the ends of the list.

- header and trailer are **sentinels**
- Nodes JFK, PVD, SFO are **real data nodes**
- Each node points **forward (next)** and **backward (prev)**

◆ PART E: Operations

? Prompt

How do we insert a new node between two existing nodes?

✓ Answer

1. Let `pred` = node before insertion, `succ` = node after
2. Create `newest = Node(element)`
3. `newest.prev = pred`
4. `newest.next = succ`
5. `pred.next = newest`
6. `succ.prev = newest`

✓ Works for all positions **including beginning and end** with sentinels.

? Prompt

How do we remove a node from a DLL?

✓ Answer

1. Let `node` = node to remove
2. `pred = node.prev`
3. `succ = node.next`
4. `pred.next = succ`
5. `succ.prev = pred`
6. Optional: clean `node.next` and `node.prev` to `None`
7. Decrement size

✓ Works in **O(1)** time if node reference is known.

◆ PART F: Comparison With Singly Linked List

Feature	SLL	DLL
Prev pointer	✗	✓
Remove tail	O(n)	O(1)
Remove arbitrary node	O(n)	O(1)

Insert anywhere	$O(n)$	$O(1)$
Memory	Less	Extra for prev

Inserting and Deleting in a Doubly Linked List

(with Header & Trailer Sentinels)

◆ PART A: Core Idea

? Prompt

Where does insertion take place in a doubly linked list with sentinels?

✓ Answer

In a doubly linked list with **header and trailer sentinels**, every insertion happens between two existing nodes.

These two nodes can be:

- `header` and first data node (insert at front)
- Two interior nodes
- Last data node and `trailer` (insert at end)

✓ Because sentinels always exist, **no special cases are needed**.

? Prompt

Why do header and trailer sentinels simplify list operations?

✓ Answer

Because:

- Every real node always has **two neighbors**
- No node is ever “at the edge” logically
- Same code works for:
 - front insertion
 - back insertion
 - middle insertion
 - deletion anywhere

◆ PART B: Insertion in a Doubly Linked List

? Prompt

Explain the general idea of inserting a node in a DLL.

✓ Answer

To insert a new node:

1. Identify two neighboring nodes:
 - a. `pred` (predecessor)
 - b. `succ` (successor)
2. Create a new node
3. Link the new node between `pred` and `succ`
4. Update both neighbors' pointers

? Prompt

Write the steps for inserting a node between two nodes in a DLL.

✓ Answer

```
Let pred = node before insertion
```

```
Let succ = node after insertion
```

```
new = Node(e)
```

```
new.prev = pred
```

```
new.next = succ
```

```
pred.next = new
```

```
succ.prev = new
```

✓ **Time Complexity:** $O(1)$

Diagram: General Insertion (Figure 7.11)

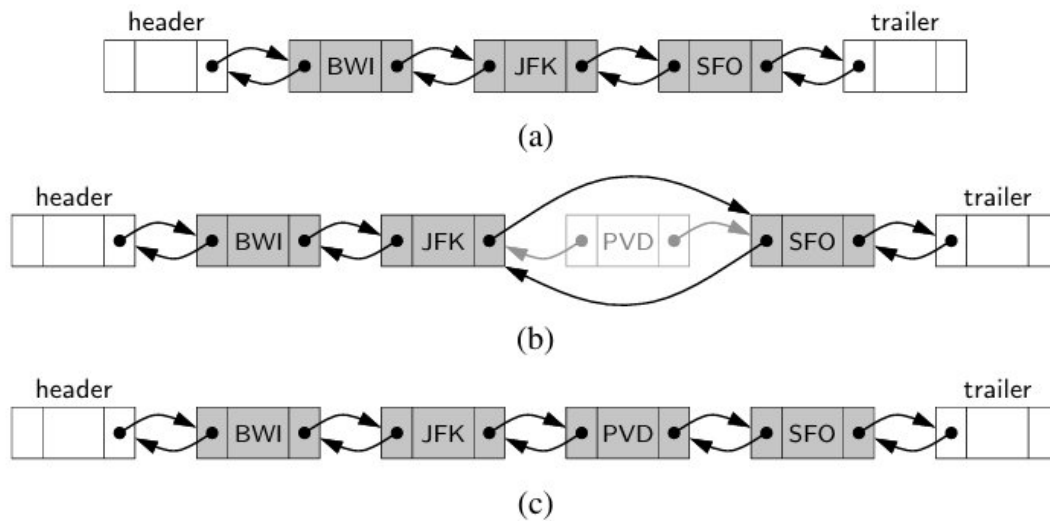


Figure 7.11: Adding an element to a doubly linked list with header and trailer sentinels: (a) before the operation; (b) after creating the new node; (c) after linking the neighbors to the new node.

◆ PART C: Inserting at the Front

? Prompt

How is insertion at the front done in a doubly linked list with sentinels?

✓ Answer

Insertion at the front means:

- Insert the new node **between** `header` and `header.next`

No special case needed — even if the list is empty.

? Prompt

Give the steps for inserting an element at the front of a DLL.

✓ Answer

```
pred = header
```

```
succ = header.next
```

```
new = Node(e)
```



```

new.prev = pred
new.next = succ
pred.next = new
succ.prev = new

```

Diagram: Front Insertion (Figure 7.12)

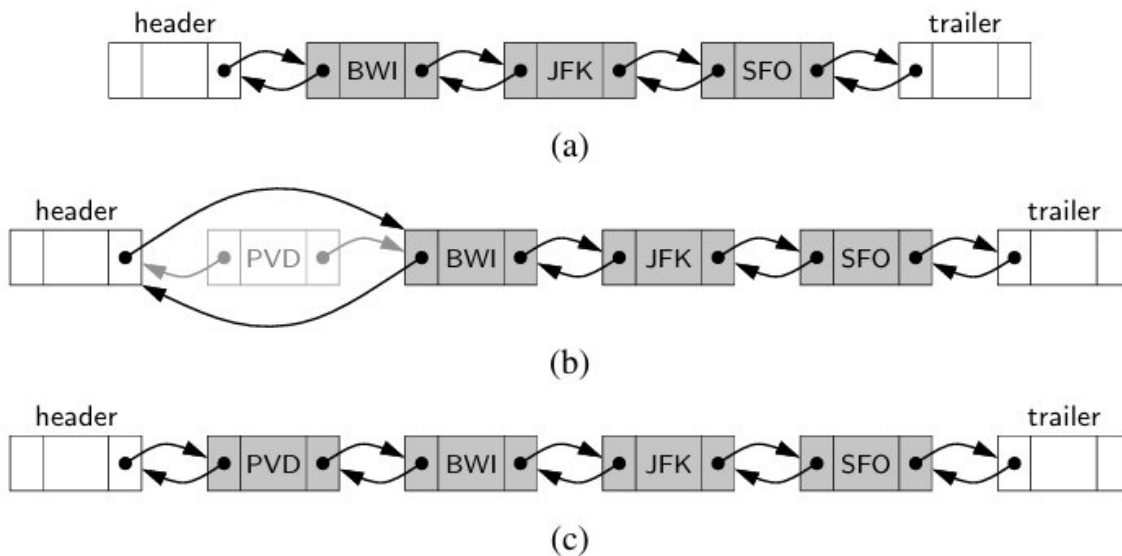


Figure 7.12: Adding an element to the front of a sequence represented by a doubly linked list with header and trailer sentinels: (a) before the operation; (b) after creating the new node; (c) after linking the neighbors to the new node.

◆ PART D: Deletion in a Doubly Linked List

? Prompt

Explain the idea of deleting a node from a DLL.

✓ Answer

To delete a node:

1. Identify its two neighbors
2. Link the neighbors directly to each other
3. The target node becomes unreachable
4. System reclaims the node (garbage collection)

? Prompt

Write the steps for deleting a node from a DLL.

✓ Answer

```
Let node = node to delete
```

```
pred = node.prev
```

```
succ = node.next
```

```
pred.next = succ
```

```
succ.prev = pred
```

✓ Time Complexity: $O(1)$

Diagram: Deletion (Figure 7.13)

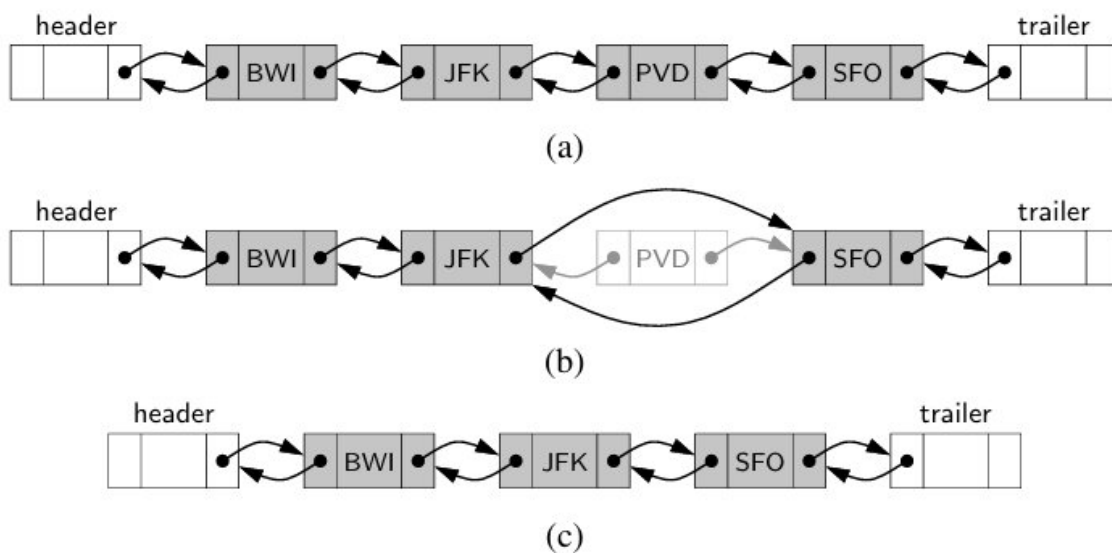


Figure 7.13: Removing the element PVD from a doubly linked list: (a) before the removal; (b) after linking out the old node; (c) after the removal (and garbage collection).

◆ PART E: Why Deletion is Efficient

? Prompt

Why can deletion be done in constant time in a doubly linked list?

✓ Answer

Because:

- Each node stores both `prev` and `next`
- The neighbors of a node are directly accessible
- No traversal is required

This is **not possible** in a singly linked list.

Basic Implementation of a Doubly Linked List

◆ PART A: Why We Need `DoublyLinkedListBase`

? Prompt

Why are integer indexes not suitable for linked lists?

✓ Answer

Unlike arrays, linked lists:

- Do **not** support direct access to the *j*th element
- Require traversal from the head to reach a position
- Access by index takes **O(n)** time

So, indexes are inefficient for describing positions in linked lists.

? Prompt

How is a position identified in a linked list?

✓ Answer

Positions are identified using **node references**, not indexes.

If we know the node:

- We can insert before or after it
- We can delete it directly
 - ✓ All in **O(1)** time

? Prompt

Why is `DoublyLinkedListBase` designed as a nonpublic (base) class?

✓ Answer

Because:

- It exposes low-level node manipulation
- Users should not directly access nodes
- It serves as a **foundation** for safer abstractions

◆ PART B: Node Class in a Doubly Linked List

? Prompt

What are the components of a doubly linked list node?

✓ Answer

Each node stores:

- `_element` → user data
- `_prev` → reference to previous node
- `_next` → reference to next node

Node class (Code Fragment 7.11)

```
class Node:

    """Lightweight, nonpublic class for storing a doubly linked node."""

    __slots__ = '_element', '_prev', '_next'

    def __init__(self, element, prev, next):

        self._element = element

        self._prev = prev

        self._next = next
```

★ Why `__slots__`?

- Reduces memory usage
- Prevents creation of extra attributes
- Important for large linked structures

◆ PART C: Constructor and Sentinels

? Prompt

What are header and trailer nodes used for?

✓ Answer

They are **dummy (sentinel) nodes** that:

- Do not store real elements
- Mark the beginning and end of the list
- Eliminate boundary special cases

? Prompt

Explain the constructor of `DoublyLinkedListBase`.

✓ Answer

```
def __init__(self):  
    self._header = self.Node(None, None, None)  
    self._trailer = self.Node(None, None, None)  
    self._header._next = self._trailer  
    self._trailer._prev = self._header  
    self._size = 0
```

🔍 What happens here?

- Header points to trailer
- Trailer points back to header
- List starts empty
- Sentinels never change

◆ PART D: Size and Empty Check

? Prompt

What does `__len__()` do?

✓ Answer

Returns the number of real elements (not counting sentinels).

```
def __len__(self):  
    return self._size
```

? Prompt

How do we check if the list is empty?

✓ Answer

```
def is_empty(self):  
    return self._size == 0
```

◆ PART E: Insertion Utility (`_insert_between`)

? Prompt

What is the role of `_insert_between`?

✓ Answer

It inserts a new element **between two existing nodes** in $O(1)$ time.

Used internally by subclasses.

Code: `_insert_between`

```
def _insert_between(self, e, predecessor, successor):  
    newest = self.Node(e, predecessor, successor)  
    predecessor._next = newest  
    successor._prev = newest
```

```
self._size += 1

return newest
```

? Prompt

Explain `_insert_between` step by step.

✓ Answer

1. Create a new node
2. Set its `prev` and `next`
3. Update predecessor's `next`
4. Update successor's `prev`
5. Increase size
6. Return the new node

◆ PART F: Deletion Utility (`_delete_node`)

? Prompt

What does `_delete_node` do?

✓ Answer

Removes a **non-sentinel node** from the list in $O(1)$ time.

Code: `_delete_node`

```
def _delete_node(self, node):

    predecessor = node._prev

    successor = node._next

    predecessor._next = successor

    successor._prev = predecessor

    self._size -= 1

    element = node._element

    node._prev = node._next = node._element = None

    return element
```

? Prompt

Explain `_delete_node` step by step.

✓ Answer

1. Identify predecessor and successor
2. Link predecessor directly to successor
3. Decrease size
4. Save element to return
5. Clear node fields (deprecate node)
6. Return stored element

◆ PART G: Why Deprecate the Node?

? Prompt

Why are the deleted node's fields set to `None`?

✓ Answer

- Helps Python's garbage collector
- Removes unnecessary references
- Prevents accidental reuse
- Allows detection of deprecated nodes